

Audion Hub Manager

Audion Hub Manager — локальный portable/NiceGUI-комбайн для проекта, Markdown-документации, Git, VS Code и безопасного зеркалирования.

Главная модель:

Source = полный проект и источник истины
Hub Data = фильтрованное техническое зеркало с Git
Docs = необязательный человекочитаемый слой документации

Hub Manager нужен для того, чтобы проект можно было видеть, проверять, коммитить и восстанавливать без перетаскивания runtime, кешей, логов, сборочных артефактов и локальных секретов.

Как выбирается проект

Dropdown **Проект** выбирает запись из **config/projects.json**. Это не поиск по дереву и не выбор одной папки внутри уже открытого дерева. Он переключает весь активный комплект:

source_path → Project/Source layer
projection_path → Hub Data / Mirror layer
docs_path → Docs layer
profile → правила MIRROR и commit allowlist
default_branch → ветка Git-команд

Относительные пути в **projects.json** считаются от корня Hub Manager. Поэтому portable-сборка может хранить для себя **"source_path": "."**, а для проектов внутри того же переносимого дерева — короткие относительные пути. Если путь находится вне дерева менеджера, Hub Manager оставляет абсолютное значение.

Панель **Структура** содержит три переключателя слоя выбранного проекта. Badge рядом с заголовком показывает активный слой и его путь:

Project = живой source_path выбранного проекта
Mirror = профильное зеркало проекта в Hub Data
Docs = читающая Markdown/text папка проекта

Если есть общая папка с двадцатью проектами, например `S:/TOOLS/Apps/`, её не надо делать одним проектом. Нажмите `Rebuild dropdown` / `Скан проектов`: Hub Manager найдёт реальные вложенные корни проектов, включая двойные папки вида `Project/Project`, и добавит отдельные записи в `projects.json`. После этого проекты переключаются dropdown-ом по одному, а MIRROR/commit выполняются для текущей активной записи.

Если в реестре остались старые demo/sample записи, битые `source_path` или дубликаты, используйте `Support -> Clean projects.json`. Кнопка чистит только `config/projects.json`: папки проектов, Source, Hub Data и Docs она не удаляет.

Документация

- `docs/README_RU.md` / `docs/README_EN.md` — обзор проекта.
- `docs/USER_GUIDE_RU.md` / `docs/USER_GUIDE_EN.md` — пользовательский сценарий.
- `docs/TECH_SPEC_RU.md` / `docs/TECH_SPEC_EN.md` — техническая спецификация.
- `docs/GIT_WIKI_HUB_MANAGER_RU.md` / `docs/GIT_WIKI_HUB_MANAGER_EN.md` — подробная Git-work wiki.
- `AGENTS_RU.md` / `AGENTS_EN.md` — контракт для Codex/агентов.
- `Docs/` — пользовательская и стратегическая документация.
- PDF-копии генерируются только по явному запросу и не хранятся как обычное зеркало документации.

Markdown является первичным форматом. PDF — разовый экспорт для передачи или архива, а не отслеживаемое дерево дублей.

Что умеет приложение

- Вести реестр проектов из `config/projects.json`.
- Сканировать общую папку с проектами и добавлять найденные проекты в реестр.
- Показывать `SOURCE`-бейдж по всем проектам после `Batch Git status`: `projects / clean / dirty / errors`.

- Выполнять `SOURCE ACTIONS` для реестра: rebuild dropdown, batch preview MIRROR, Clone Source, batch safety, batch verify, batch Git status, combined workspace.
- Выполнять `PROJECT ACTIONS` для текущего выбранного объекта дерева: refresh, load to Editor, open in VS Code, copy relative/full path.
- Открывать текущий проект через `Open Project`, а также Mirror, Docs Folder, VS Code, Terminal и Git. `Terminal` открывает терминал в Source, `Git` открывает Git-root terminal и сразу выполняет `git status --short`.
- Чистить `projects.json` от отсутствующих Source-путей и дублей через `Support -> Clean projects.json`.
- Строить Hub Projection по профилю из `config/projection_profiles.json`.
- Делать dry-run и apply MIRROR с защитой Source.
- Показывать слои Project/GIT COPY/Docs в панели `Структура`.
- Выполнять прозрачные Git-команды и показывать точный terminal output.
- Закрывать почти весь повседневный Git lifecycle: init/status/inspect, diff, stage/restore, commit, tag, remote sync, branch/switch, stash, history/graph, recovery, maintenance, clone into Source и ручной command cache. Редкие или опасные операции остаются явными шаблонами.
- Держать главные локальные Git-команды в `Quick -> GIT LOCAL`: `init`, `status`, `root`, `log --oneline`, `reflog`. `user config` находится в Auth-блоке внутри `Remote`, `config list` — в Git backup/maintenance, Graph-история находится во вкладке `History`.
- Использовать `Branch` для branch status, `branch -vv`, `switch`, `switch -c`, tags, compare branches, `merge --no-ff`, `revert`, `cherry-pick`, stash и abort/rebase templates.
- Готовить читаемые commit/checkpoint через Basket.
- Открывать Markdown/text файлы во встроенном Editor с icon-only toolbar: load/save/paste/copy/clear/VS Code/expand.
- Показывать двухрядную сетку правых вкладок: Quick, Branch, Editor, Diff, Storage; Remote, Basket, Reader, History и Details. Safety Scan живёт в Storage.
- Показывать Material-иконки в заголовках правых вкладок и delayed tooltips на command buttons и tabs.
- Использовать `Remote` для сборки remote URL, recent-кэша полей, `push origin`, `pull --ff-only`, `fetch --all --prune`, `remote -v`, `push all remotes`, `apply remotes.json`, `origin push URLs` и Auth setup.
- Проверять BLAKE3 backend и делать read-only Verify Mirror для Source <-> Hub Data.
- Проверять внешнюю Git-аутентификацию без хранения токенов.
- Помогать восстановить проект из Hub Data, если Source был удалён.

Запуск

GUI:

```
launcher_gui.cmd
```

CLI smoke:

```
launcher_cli.cmd --mirror-preview demo_local --json
launcher_cli.cmd --mirror-apply demo_local --json
launcher_cli.cmd --mirror-apply demo_local --apply --json
```

Если portable runtime отсутствует:

```
install\Build_Portable_Env.cmd
```

Настройка проекта

Проекты описаны в:

```
config/projects.json
```

Пример:

```
{
  "id": "audion_hub_manager",
  "title": "Audion Hub Manager",
  "source_path": ".",
  "projection_path": "S:/Audion/Hub Data/Audion Hub Manager",
  "docs_path": "S:/Audion/Docs/Projects/Audion Hub Manager",
  "profile": "audion_python_project_projection",
  "default_branch": "main"
}
```

source_path — живой полный проект. Относительные значения считаются от корня Hub Manager. MIRROR не пишет туда.

`projection_path` — фильтрованное Hub Data зеркало. Оно может пересоздаваться и иметь собственный `.git`.

`docs_path` — необязательный docs-view для Markdown/text чтения.

Правила безопасности

1. Source — источник истины.
2. Hub Data — производное зеркало, его можно удалить и пересобрать.
3. MIRROR не изменяет Source.
4. `.git/**` защищён.
5. Реальный MIRROR apply требует явного намерения.
6. Коммиты Hub Manager должны использовать тот же allowlist, что и профиль Hub.
7. Не хранить токены, пароли, приватные ключи и локальные пути в shared config.
8. Machine-local настройки держать в `*.local.json`, `.env` и других excluded файлах.
9. BLAKE3 manifests/check reports пишутся только в `logs/<project_id>/`.
10. Docs/Obsidian/LogSeq/VS Code docs-папки не хэшируются отдельно: их техническая каноническая копия уже находится в Hub Data.

Verify Mirror — read-only проверка Source ↔ Hub Data по активному профилю. Манифесты строятся в памяти, результат пишется одним датированным JSON в `logs/<project_id>/`. Source, Hub Data и Docs не получают служебных manifest files.

Git

Рекомендуемая модель:

```
Full Project Source/  
  .git/  
  
Hub Data/<project>/  
  .git/
```

Source Git нужен для живой разработки, VS Code и агентов.

Hub Git нужен как чистая история фильтрованного технического зеркала.

Hub Manager не должен делать широкий `git add .`. Он должен stage/commit только те файлы, которые допустил бы активный projection profile.

Remote и Auth

Hub Manager не является хранилищем секретов.

Remote names, логины/группы и имена репозиторий вносятся во вкладку `Remote`. Recent-select кэширует до 20 значений на поле и применяет значение только после явного выбора, поэтому ввод или вставка нового текста не перекрывается подсказками кэша.

Используйте внешние инструменты:

```
gh auth login
glab auth login
ssh -T git@github.com
ssh -T git@gitlab.com
```

HTTPS credentials должны жить в Git Credential Manager или OS credential store. SSH ключи должны жить в SSH/ssh-agent.

Платформа и вид URL выбираются кнопками в верхнем ряду `Remote`, без dropdown. Для `Forgejo`, `Gitea` и `Свой сервер` появляются поля адреса и SSH-порта, а список известных инстансов лежит в `config/forgejo_hosts.json` — без токенов.

Учётная запись Forgejo/Gitea подключается обычным для этих серверов способом: personal access token из `Settings -> Applications`. Hub Manager проверяет токен запросом `/api/v1/user` и передаёт его вашему Git credential helper; в файлы проекта он не записывается. После этого `git push` берёт токен из хранилища сам, а API-кнопки показывают учётную запись, список репозиторий и позволяют создать новый. Подробности и обоснование выбора — в `docs/GIT_AUTH_STRATEGY.md`.

Проверка

```
python -m compileall -q system_core
python -m pytest -q tests
python system_core\ui_nicegui\app.py --smoke
```

Документированный baseline: `121 passed`.

Восстановление

Если Source удалён, но Hub Data жив:

```
robocopy "<Hub Data>\Audion Hub Manager" "<portable-root>\Audion Hub Manager" /E  
/COPY:DAT /DCOPY:DAT /R:1 /W:1 /XJ
```

После этого пересоберите runtime или release-артефакты при необходимости. PDF генерируйте только для явной передачи или архива.

Короткая формула проекта:

```
Source stays whole.  
Hub stays reviewable.  
Docs stays readable.  
Git stops being scary.
```